Linear Equations and Solutions Using NumPy

Your Name

December 21, 2024

Linear Equations - General Case

System of Linear Equations:

$$
\begin{aligned}
a_{11}x_1 + a_{12}x_2 + \ldots + a_{1n}x_n &= b_1 \\
a_{21}x_1 + a_{22}x_2 + \ldots + a_{2n}x_n &= b_2 \\
&\vdots \quad \vdots \\
a_{m1}x_1 + a_{m2}x_2 + \ldots + a_{mn}x_n &= b_m
\end{aligned}
$$

Linear Equations - General Case

System of Linear Equations:

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \ldots + a_{1n}x_n &= b_1 \\ a_{21}x_1 + a_{22}x_2 + \ldots + a_{2n}x_n &= b_2 \\ &\vdots \quad \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \ldots + a_{mn}x_n &= b_m \end{aligned}$$

Matrix Representation:

$$A\mathbf{x} = \mathbf{b}$$

Linear Equations - General Case

System of Linear Equations:

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \ldots + a_{1n}x_n &= b_1 \\ a_{21}x_1 + a_{22}x_2 + \ldots + a_{2n}x_n &= b_2 \\ &\vdots \quad \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \ldots + a_{mn}x_n &= b_m \end{aligned}$$

Matrix Representation:

$$A\mathbf{x} = \mathbf{b}$$

where:

- A is an $m \times n$ matrix of coefficients.
- $\mathbf{x}$ is an $n \times 1$ vector of unknowns.
- $\mathbf{b}$ is an $m \times 1$ vector of constants.

General Solution Using Inverse Matrix

Linear System:

$$A\mathbf{x} = \mathbf{b}$$

General Solution Using Inverse Matrix

Linear System:

$$A\mathbf{x} = \mathbf{b}$$

Solution Using Inverse:

- If A is **square (n n)** and **invertible (non-singular)**, we can find:

$$\mathbf{x} = A^{-1}\mathbf{b}$$

- A^{-1} is the **inverse matrix**, and it satisfies:

$$AA^{-1} = I$$

where I is the **identity matrix**.

General Solution Using Inverse Matrix

Linear System:

$$A\mathbf{x} = \mathbf{b}$$

Solution Using Inverse:

- If A is **square (n n)** and **invertible (non-singular)**, we can find:

$$\mathbf{x} = A^{-1}\mathbf{b}$$

- A^{-1} is the **inverse matrix**, and it satisfies:

$$AA^{-1} = I$$

where I is the **identity matrix**.

Conditions:

- A must be **square** ($n \times n$).
- $\det(A) \neq 0$ (non-singular).

General Solution Using Inverse Matrix

Linear System:

$$A\mathbf{x} = \mathbf{b}$$

Solution Using Inverse:

- If A is **square (n n)** and **invertible (non-singular)**, we can find:

$$\mathbf{x} = A^{-1}\mathbf{b}$$

- A^{-1} is the **inverse matrix**, and it satisfies:

$$AA^{-1} = I$$

where I is the **identity matrix**.

Conditions:

- A must be **square** ($n \times n$).
- $\det(A) \neq 0$ (non-singular).

Example:

$$A = \begin{bmatrix} 2 & 1 \\ 1 & -1 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 4 \\ 1 \end{bmatrix}$$

$$\mathbf{x} = A^{-1}\mathbf{b}$$

Solution Types and Rank

Types of Solutions:

- **Unique Solution:** If the rank of A equals the number of unknowns (n).
- **Infinite Solutions:** If the rank of A equals the rank of the augmented matrix $[A|\mathbf{b}]$ but is less than n.
- **No Solution:** If the rank of A is less than the rank of the augmented matrix $[A|\mathbf{b}]$.

Solution Types and Rank

Types of Solutions:

- **Unique Solution:** If the rank of A equals the number of unknowns (n).
- **Infinite Solutions:** If the rank of A equals the rank of the augmented matrix $[A|\mathbf{b}]$ but is less than n.
- **No Solution:** If the rank of A is less than the rank of the augmented matrix $[A|\mathbf{b}]$.

Key Concept: Rank

- Rank determines the **number of linearly independent rows or columns**.
- NumPy provides:

  np.linalg.matrix_rank(A)

  to compute the rank of a matrix.

Solution Types and Rank

Types of Solutions:

- **Unique Solution:** If the rank of A equals the number of unknowns (n).
- **Infinite Solutions:** If the rank of A equals the rank of the augmented matrix $[A|\mathbf{b}]$ but is less than n.
- **No Solution:** If the rank of A is less than the rank of the augmented matrix $[A|\mathbf{b}]$.

Key Concept: Rank

- Rank determines the **number of linearly independent rows or columns**.
- NumPy provides:

  np.linalg.matrix_rank(A)

  to compute the rank of a matrix.

Example:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, \quad \text{Rank}(A) = 2$$

NumPy Example: Unique Solution

Example: Solve $A\mathbf{x} = \mathbf{b}$

```
1  import numpy as np
2
3  # Coefficient matrix A
4  A = np.array([[2, 1],
5                [1, -1]])
6
7  # Constant vector b
8  b = np.array([4, 1])
9
10 # Solve the system
11 x = np.linalg.solve(A, b)
12 print("Solution: ", x)
```

NumPy Example: Unique Solution

Example: Solve $A\mathbf{x} = \mathbf{b}$

```python
import numpy as np

# Coefficient matrix A
A = np.array([[2, 1],
              [1, -1]])

# Constant vector b
b = np.array([4, 1])

# Solve the system
x = np.linalg.solve(A, b)
print("Solution: ", x)
```

Output:

$$\mathbf{x} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$$

NumPy Example: Infinite and No Solutions

1. Infinite Solutions:

```
A = np.array([[1, 2],
              [2, 4]])
b = np.array([3, 6])

rank_A = np.linalg.matrix_rank(A)
aug = np.hstack([A, b.reshape(-1, 1)])
rank_aug = np.linalg.matrix_rank(aug)

print("Rank of A:", rank_A)
print("Rank of Augmented Matrix:", rank_aug)
```

NumPy Example: Infinite and No Solutions

1. Infinite Solutions:

```
A = np.array([[1, 2],
              [2, 4]])
b = np.array([3, 6])

rank_A = np.linalg.matrix_rank(A)
aug = np.hstack([A, b.reshape(-1, 1)])
rank_aug = np.linalg.matrix_rank(aug)

print("Rank of A:", rank_A)
print("Rank of Augmented Matrix:", rank_aug)
```

2. No Solution:

```
A = np.array([[1, 2],
              [2, 4]])
b = np.array([3, 7])

rank_A = np.linalg.matrix_rank(A)
aug = np.hstack([A, b.reshape(-1, 1)])
rank_aug = np.linalg.matrix_rank(aug)
```

Applications of Linear Systems

Real-World Applications:

- **Engineering Problems**: Solving circuit systems using **Kirchhoff's Laws**.
- **Data Analysis**: Regression analysis and linear modeling.
- **Computer Graphics**: Transformations and projections.
- **Machine Learning**: Optimization problems like **least squares**.

Applications of Linear Systems

Real-World Applications:

- **Engineering Problems**: Solving circuit systems using **Kirchhoff's Laws**.
- **Data Analysis**: Regression analysis and linear modeling.
- **Computer Graphics**: Transformations and projections.
- **Machine Learning**: Optimization problems like **least squares**.

Next Steps: - Use these concepts to solve real-world problems. - Extend to **singular value decomposition (SVD)** and **eigenvalue problems**.